

Amlogic_NPU_LiteRT_Guide

Revision: 0.1

Release Date: 2026-06-25

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision history

Version	Change Description
v0.1 (2026-06-25)	This is the First Version.

Contents

1 LiteRT Overview

- 1.1 Introduction
- 1.2 Key Features
- 1.3 Amlogic and LiteRT Integration Architecture
- 1.4 Interpreter API
- 1.5 CompiledModel API
- 1.6 Interpreter vs. CompiledModel Comparison
- 1.7 AML Delegate Introduction

2. Platform Deployment Overview

- 2.1 Supported Platforms and Delivery Forms
- 2.2 API and Platform Selection
 - 2.2.1 Platform Selection
 - 2.2.2 API Selection
- 2.3 Pre-Deployment Checks
 - 2.3.1 Hardware and Versions
 - 2.3.2 Operators and Data Types
 - 2.3.3 Pre-Deployment Checklist
- 2.4 Demo and Example Index

3. Android Deployment

- 3.1 Introduction
- 3.2 Support Status
- 3.3 Interpreter API Deployment Process
 - 3.3.1 Import LiteRT AAR
 - 3.3.2 Edit `app/build.gradle`
 - 3.3.3 Initialize Interpreter and Attach AML Delegate
 - 3.3.4 Device-Side Debugging (Interpreter Local Executable Cases)
 - 3.3.5 Demo and Complete Project
- 3.4 CompiledModel API Deployment Process
 - 3.4.1 Gradle Dependencies
 - 3.4.2 Kotlin Quick Start
 - 3.4.3 Differences from Interpreter (Summary)
 - 3.4.4 Device-Side Additional Libraries (Example)
 - 3.4.5 Demo and Complete Project (TODO)

4. Linux Deployment

- 4.1 Introduction
- 4.2 Support Status
- 4.3 Interpreter API Deployment
 - 4.3.1 Interpreter C API Interpreter C API Interface Description
 - 4.3.2 Deployment Process
 - 4.3.3 Application Project Integration (C/C++)
 - 4.3.4 Validation Suggestions
- 4.4 CompiledModel API Deployment (TODO)

1 LiteRT Overview

LiteRT (Lite Runtime, successor to TensorFlow Lite) is Google's unified inference framework for edge AI. This document is based on the [Google LiteRT official documentation](#), summarizing integration and deployment information related to Android and Linux, and explaining how Amlogic's **AML Delegate** integrates with LiteRT to utilize ADLA NPU.

Official Resource	Link
LiteRT Overview	developers.google.com/edge/litert
Android Development	developers.google.com/edge/litert/android
Sample Applications	LiteRT Sample Apps on GitHub

1.1 Introduction

LiteRT evolved from TensorFlow Lite and is designed for edge platforms such as Android, Linux, and embedded systems, providing model loading, hardware acceleration, and inference execution capabilities. On the application side, `.tflite` serves as the unified model format, and subgraphs can be offloaded to accelerators like GPU/NPU through **Delegates**.

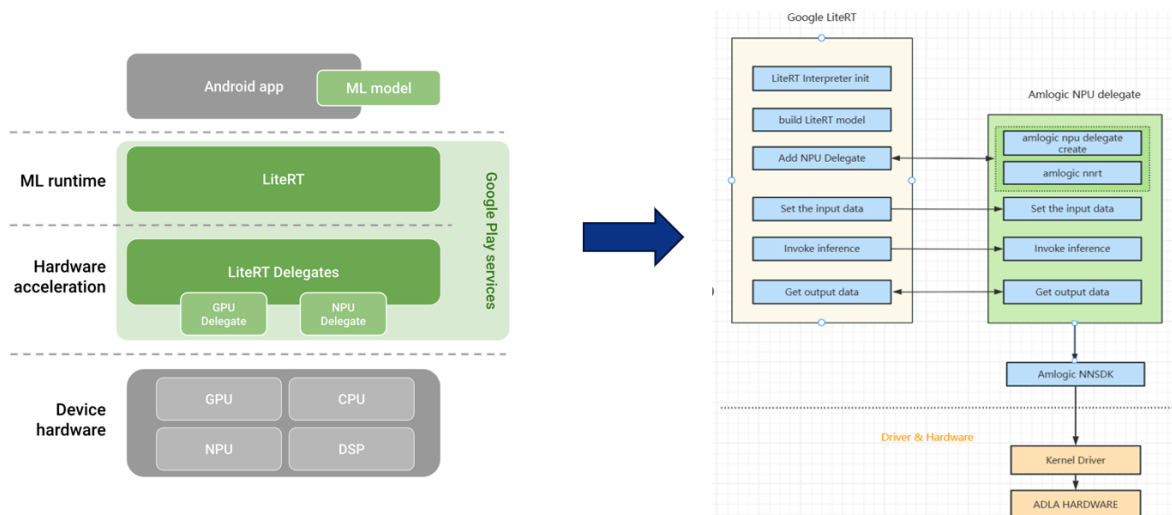
On Amlogic platforms, this document focuses on explaining how to integrate LiteRT with **AML Delegate** to complete NPU inference deployment in Android APKs or Linux native services.

1.2 Key Features

- **On-Device Optimizations:** Low latency, privacy protection, offline capability, small footprint, low power consumption.
- **Cross-Platform:** Supports Android, Linux (including embedded distributions like Yocto / Debian), iOS, etc.
- **Multi-Framework Model Support:** Models from TensorFlow, PyTorch, JAX, etc., can be converted to `.tflite` and run.
- **Multi-Language SDKs:** Java/Kotlin, C++, Python, etc.
- **Hardware Acceleration:** Leverages Delegates or CompiledModel accelerator options to connect to CPU / GPU / NPU backends.

1.3 Amlogic and LiteRT Integration Architecture

The diagram below illustrates the integration relationships between the LiteRT runtime, Amlogic AML Delegate, NNSDK, ADLA, and other modules (**to be supplemented**):



1.4 Interpreter API

The **Interpreter API** belongs to the LiteRT **V1.x series** (e.g., v1.2.0, v1.4.2), maintained by Google as the **basic inference layer**, ensuring backward compatibility for existing TFLite and LiteRT applications.

Item	Description
Official Position	Basic inference API, maintained for backward compatibility
Model Format	<code>.tflite</code> (FlatBuffers)
Hardware Acceleration	Accessed via the Delegate mechanism (<code>addDelegate</code> / <code>ModifyGraphWithDelegate</code>), supporting backends like CPU / GPU / NPU (e.g., AML Delegate).

Item	Description
Typical Usage	Load model → Configure input/output tensors → (Optional) register Delegate → Call <code>run()</code> or <code>runForMultipleInputsOutputs()</code> for synchronous inference.

Amlogic Call Path:

App → LiteRT Interpreter → AML Delegate → Runtime → NNSDK → ADLA Driver → NPU

1.5 CompiledModel API

The **CompiledModel API** belongs to the LiteRT **V2.x series** (e.g., v2.1.0, v2.1.5), defined by Google as the **new generation high-performance inference standard**. This API unifies the hardware acceleration path for CPU / GPU / NPU, and provides asynchronous execution capabilities and a simplified device selection mechanism.

Item	Description
Official Position	Modern standard for high-performance inference
Model Format	<code>.tflite</code>
Hardware Acceleration	Explicitly specified via <code>CompiledModel.Options(Accelerator.CPU/GPU/NPU)</code> , also supports vendor-specific extension backends.
Typical Usage	Create <code>CompiledModel</code> instance → Allocate input/output buffers (<code>createInputBuffers / createOutputBuffers</code>) → Fill data → Execute inference (<code>run</code>).

Amlogic Call Path:

APP → LiteRT CompiledModel → Dispatch / Compiler Plugin → Runtime → NNSDK → ADLA Driver → NPU

1.6 Interpreter vs. CompiledModel Comparison

Comparison Aspect	Interpreter	CompiledModel
Representative Versions	v1.2.0 ~ v1.4.2 (Latest: v1.4.2)	v2.0.3 ~ v2.1.5 (Latest: v2.1.5)
Core API	<code>Interpreter + addDelegate</code>	<code>CompiledModel.create + run</code>
NPU Integration	AML Delegate (<code>AmlDelegate</code>)	Dispatch / Compiler Plugin + AML Extension
Backward Compatibility	Compatible with existing TFLite code	New API, requires adaptation to V2 paradigm
Amlogic Maturity	Current main delivery path	Capabilities evolve with versions
Google Recommendation	For existing apps, quick migration	** Priority for new projects**

For the correspondence between versions and supported APIs, refer to [Google LiteRT Android version notes](#). **Platform and API selection suggestions are in Section 2.2.**

1.7 AML Delegate Introduction

LiteRT models use the FlatBuffers format (`.tflite`). During inference, a **Delegate** is responsible for **offloading** acceleratable computational subgraphs to hardware backends like GPU, DSP, or NPU. Parts not delegated are executed by the CPU.

The **AML Delegate** is Amlogic's LiteRT hardware delegate implementation for the **ADLA NPU**. It forwards the subgraphs partitioned by LiteRT to the NNSDK / ADLA runtime for acceleration. Hardware constraints, operator support, and version compatibility requirements for this delegate are detailed in Section 2.3.

2. Platform Deployment Overview

2.1 Supported Platforms and Delivery Forms

Platform	Typical Scenarios	Delivery Form
Android	APK / System Apps, Java business layer integration	Maven <code>com.google.ai.edge.litert + liteRT_aml_delegate.aar</code>

Platform	Typical Scenarios	Delivery Form
Linux	Embedded services, CLI/daemon process verification	<code>libLiteRt.so</code> + AML Delegate <code>.so</code> + Native executable cases; LiteRT / Delegate libraries can be compiled into images or pushed to the target filesystem.

This document divides deployment into two paths based on the **target operating system**: Android and Linux. Both share the same NPU acceleration capabilities at the **Interpreter + AML Delegate** level, with differences mainly in integration interfaces and delivery artifact forms.

Comparison Aspect	Android Deployment	Linux Deployment
Application Language	Java / Kotlin(can be supplemented with JNI)	C / C++
Interpreter	<code>liteRT_aml_delegate.aar</code> + Maven <code>litert</code> ; <code>AmlDelegate</code> + <code>Interpreter</code>	<code>libLiteRt.so</code> + AML Delegate related libraries; Interpreter C API ; <code>aml_delegate_test_bin</code>
CompiledModel	Requires additional integration of Dispatch / Compiler Plugin dynamic libraries	<code>libLiteRTDispatchAml.so</code> , <code>libLiteRtCompilerPluginAml.so</code> ; <code>aml_CompiledModel_benchmark</code>
Model Deployment Path	<code>assets</code> , application private directory, or specified absolute path	Filesystem absolute path (e.g., <code>/data/...</code> , working directory)

On Android, the AML Delegate is delivered as an **AAR**; on Linux, the main forms are **shared libraries + executable test cases**.

2.2 API and Platform Selection

2.2.1 Platform Selection

- **Choose Android**: Business logic within APK, requires Java/Kotlin integration, or needs to cooperate with official `com.google.ai.edge.litert` dependencies.
- **Choose Linux**: No Android framework, primarily C/C++ services/command line, or requires local test cases for NPU path validation during bring-up / CI.

2.2.2 API Selection

Select the appropriate inference API based on the project phase:

- **Existing TFLite / Interpreter project requiring quick NPU integration** → Prioritize **Interpreter API + AML Delegate**.
- **New project requiring unified CPU/GPU/NPU acceleration path and supporting asynchronous inference** → Evaluate and adopt **CompiledModel**.

Regardless of the chosen API, ensure that **LiteRT version**, **AML Delegate / AAR version**, and **ROM / BSP version** are consistent.

2.3 Pre-Deployment Checks

2.3.1 Hardware and Versions

Item	Description
Hardware	Device must be equipped with ADLA NPU , and ADLA driver must be available in ROM/BSP.
Version Matrix	** LiteRT version, AML Delegate version, ROM/BSP/driver version** must be compatible.
Platform Precision	Float32 / quantized model support varies by platform; refer to Section 1.4.2 in <code>06_Amlogic_NPU_Android_Deployment_Guide.pdf</code> .

2.3.2 Operators and Data Types

Category	Support Notes
Operators (Ops)	Refer to the operator list support.list.xlsx
Floating Point	Float32, Float16 , etc.; depends on the target platform and model configuration.
Integer/Quantization	Int8, UInt8 , etc.; supports common quantization schemes like per-layer, per-channel.

Category	Support Notes
Fallback Strategy	Subgraphs that cannot be executed by the NPU can be fallbacked to CPU by LiteRT, ensuring the entire graph can run.

2.3.3 Pre-Deployment Checklist

1. Confirm **ADLA NPU** and driver are functioning on the target board.
2. Prepare compliant `.tflite` models.
3. Align **LiteRT / AML Delegate / BSP** version combination.
4. Plan model paths and (if enabled) **compilation/loading cache** directories to avoid mixing old caches with new models.

2.4 Demo and Example Index

API	Platform	Description	Document Section
Interpreter	Android	Minimal <code>Interpreter</code> + <code>AmlDelegate</code> example	3.4 (full GitHub link to be supplemented)
Interpreter	Linux	<code>aml_delegate_test_bin</code> local case	3.4.4 , 4.4
CompiledModel	Android	Kotlin / C++ inference examples	3.5
CompiledModel	Linux	<code>main_npu</code> and other <code>CompiledModel</code> cases	4.5 (TODO)
Official Reference	—	LiteRT Sample Apps	Google Android Docs

3. Android Deployment

This chapter describes the integration methods for **LiteRT Interpreter API** and **CompiledModel API** on Android.

3.1 Introduction

This section explains how to use LiteRT with the **AML Delegate** for NPU inference within Android applications.

- **Interpreter API:** `Interpreter` + `AmlDelegate` (see ***3.3***).
- **CompiledModel API:** `CompiledModel`.

The application calls LiteRT within the APK. LiteRT parses the `.tflite` and offloads subgraphs to the ADLA NPU via the **AML Delegate**.

3.2 Support Status

Currently only adapted for Amlogic series devices with built-in ADLA NPU. For operator coverage, supported data types, and precision constraints, refer to Section 2.3.2 of this document. For performance and effect differences between native platform Float32 operations and quantized inference, refer to the companion document *06_Amlogic_NPU_Android_Deployment_Guide.pdf* for complete details.

3.3 Interpreter API Deployment Process

3.3.1 Import LiteRT AAR

Copy the `litert_aml_delegate.aar` file to the `libs/` folder in the application source code directory (create it manually if it doesn't exist).

3.3.2 Edit app/build.gradle

Edit `app/build.gradle` and include the local AAR and LiteRT official dependencies in the `dependencies` block.

```
dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar','*.aar'])
    implementation 'com.google.ai.edge.litert:litert:1.2.0'
    implementation 'com.google.ai.edge.litert:litert-support-api:1.2.0'
}
```

3.3.3 Initialize Interpreter and Attach AML Delegate

- Import the `AmlDelegate` class

```
import org.tensorflow.lite.AmlDelegate;
```

- Create an `AmlDelegate` instance

```
private static AmlDelegate amlDelegate = null;
AmlDelegate.Options options_aml = new AmlDelegate.Options();
options_aml.model_path = ""; // Fill the quotes with the directory on the board where the tflite model
is placed, e.g., "/storage/emulated/0/DCIM/yolo_int8.tflite"
amlDelegate = new AmlDelegate(options_aml);
```

- Register Delegate and build Interpreter

```
Interpreter.Options options = new Interpreter.Options();
options.addDelegate(amlDelegate);
private Interpreter tfLite;
MappedByteBuffer modelFile = loadModelFile(context.getAssets(), modelFilename);
tfLite = new Interpreter(modelFile, options);
```

- Helper method to load model (`loadModelFile`)

```
private static MappedByteBuffer loadModelFile(AssetManager assets, String modelFilename)
    throws IOException {
    AssetFileDescriptor fileDescriptor = assets.openFd(modelFilename);
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
}
```

- Execute inference

```
tfLite.runForMultipleInputsOutputs(inputArray, outputs);
```

3.3.4 Device-Side Debugging (Interpreter Local Executable Cases)

Once the build artifacts are ready, you can directly use the **Interpreter local cases** to verify the NPU path on the device (without recompiling the application project):

- Local case: `aml_delegate_test_bin`
- Runtime dependencies: `libLited.so` and AML Delegate companion shared libraries (subject to actual release package).

Push and execute:

```
adb push <path>/aml_delegate_test_bin /data/local/tmp/
adb shell
export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH
chmod +x /data/local/tmp/aml_delegate_test_bin
/data/local/tmp/aml_delegate_test_bin <args>
```

3.3.5 Demo and Complete Project

Item	Description
Example Content	Uses <code>Interpreter</code> to load <code>.tflite</code> model, attaches <code>AmlDelegate</code> , and executes inference.
Complete Project	https://github.com/Amlogic-NN/amlnn-google-ai-ecosystem/tree/main/litert

3.4 CompiledModel API Deployment Process

The `CompiledModel` API belongs to the LiteRT V2.x series (e.g., v2.1.0), providing a unified CPU / GPU / NPU acceleration path and asynchronous execution capabilities. Currently, most APK scenarios still primarily use the `Interpreter` process (Section 3.3). `CompiledModel` is suitable for new projects requiring unified multi-backend acceleration or asynchronous inference.

3.4.1 Gradle Dependencies

Include LiteRT V2.x dependencies in `app/build.gradle`:

```
dependencies {
    implementation 'com.google.ai.edge.litert:litert:2.1.0'
}
```

3.4.2 Kotlin Quick Start

The following example, adapted from Google's official documentation, demonstrates loading a model, allocating buffers, and executing inference:

```
val compiledModel = CompiledModel.create(
    "/path/to/myModel.tflite",
    CompiledModel.Options(Accelerator.CPU)) // 可替换为 GPU / NPU

val inputBuffers = compiledModel.createInputBuffers()
val outputBuffers = compiledModel.createOutputBuffers()

inputBuffers.get(0).writeFloat(input0)
compiledModel.run(inputBuffers, outputBuffers)
val output = outputBuffers.get(0).readFloat()
```

3.4.3 Differences from Interpreter (Summary)

Item	Interpreter (Main Path)	CompiledModel (Optional)
Entry API	<code>Interpreter</code> + <code>AmlDelegate</code>	<code>CompiledModel</code> and companion compile/load interfaces
Additional Dependencies	Typically included in <code>liteRT_aml_delegate.aar</code>	Requires additional Dispatch / Compiler Plugin dynamic libraries

3.4.4 Device-Side Additional Libraries (Example)

When using `CompiledModel`, companion dynamic libraries need to be pushed to the device (actual library names subject to release package):

```
adb push <path>/libLiteRt.so /data/LiteRT/
adb push <path>/libLiteRtDispatchAml.so /data/LiteRT/
adb push <path>/libLiteRtCompilerPluginAml.so /data/LiteRT/
adb push <path>/aml_CompiledModel_benchmark /data/LiteRT/
```

For specific execution results, refer to *06_Amlogic_NPU_Android_Deployment_Guide.pdf*.

3.4.5 Demo and Complete Project (TODO)

Item	Description
Example Type	<code>CompiledModel</code> loads <code>.tflite</code> model, selects <code>Accelerator</code> , and executes inference.
Official Reference	Image segmentation Kotlin/C++ Demo
Complete Project	GitHub link (TODO)

Specific API calls and model artifact formats are subject to the corresponding version's release notes.

4. Linux Deployment

This chapter describes the integration methods for **Interpreter C API** and **CompiledModel** on Linux.

4.1 Introduction

The Linux side does not go through the Android Framework. It adopts a **C/C++ + dynamic libraries + local executable cases** approach to integrate LiteRT and the AML Delegate. This is suitable for embedded Linux, solution verification, and Native service scenarios.

Interpreter Path Overview:

4.2 Support Status

Currently supported devices, operators, and model types are consistent with the Android side:

- **Devices:** Amlogic platforms with ADLA NPU, where ADLA drivers and dependent libraries are available in the Linux BSP.
- **Operators / Data Types:** See 2.3.2; platform precision differences are in *06_Amlogic_NPU_Android_Deployment_Guide.pdf*.

4.3 Interpreter API Deployment

4.3.1 Interpreter C API Interpreter C API Interface Description

The **main interface** on Linux is the **Interpreter C API**, whose responsibilities correspond to Android's `Interpreter` + `Am1Delegate`:

Capability	Description
Load Model	Load <code>.tflite</code> (Float32 / Float16 / Int8 / UInt8, etc.) from APK assets or filesystem path.
Attach Delegate	Register AML Delegate to offload delegatable subgraphs to NPU; unsupported ops fall back to CPU via LiteRT.
Execute Inference	Prepare input/output tensor buffers, call run-type interfaces.
Model Path / Cache	Supports absolute paths; if compilation cache is enabled, it must be consistent with model name/MD5 or other identifiers.

Common Interpreter Deliverables:

Type	Example	Purpose
Runtime Library	<code>libLiteRt.so</code>	LiteRT Interpreter runtime library.
Delegate Library	AML Delegate companion <code>.so</code> (name varies by release)	NPU delegate implementation.
Local Case	<code>am1_delegate_test_bin</code>	Smoke test, accuracy and performance validation for Interpreter path.

4.3.2 Deployment Process

- **Preparation:**

Confirm the following files matching the target BSP version are available locally:

- `libLiteRt.so` and AML Delegate related `.so` files.
- `am1_delegate_test_bin` (Interpreter validation case).
- Test `.tflite` model (floating point or quantized is acceptable).

- **Copy to Board Directory**

Copy the `.so` files and executable to a unified working directory on the board (example uses `/data/LiteRT/`, can be adjusted per project agreement):

```
scp libLiteRt.so root@<board>:/data/LiteRT/
scp am1_delegate_test_bin root@<board>:/data/LiteRT/
scp <model>.tflite root@<board>:/data/LiteRT/

# On the board, grant execute permission
chmod +x /data/LiteRT/am1_delegate_test_bin
```

- **Run Interpreter Local Case**

```
cd /data/LiteRT
./am1_delegate_test_bin *.tflite
```

Specific parameters are subject to the released case's instructions (common parameters include model path, input dimensions, loop counts, etc.).

4.3.3 Application Project Integration (C/C++)

在When integrating into your own Native project following the **Interpreter** path, the general steps are:

- 1. Link against `libLiteRt.so` and AML Delegate related libraries.
- 2. Load `tfLite` via **Interpreter C API** and register the AML Delegate.
- 3. Prepare input/output buffers and call inference interfaces.
- 4. Configure model absolute path, input data source, and compilation cache directory as needed.

4.3.4 Validation Suggestions

- 1. **Initialization:** Run `aml_delegate_test_bin` to confirm the NPU path is usable.
- 2. **Accuracy:** Compare CPU fallback outputs with NPU outputs (can be compared against PC golden standard or benchmark scripts).
- 3. **Performance:** Distinguish pipeline time (loading/first compilation) from runtime pure inference time.
- 4. **Stability:** Long-term loops, switching between multiple models, abnormal power recovery, etc.

4.4 CompiledModel API Deployment (TODO)

Used only when CompiledModel is required; runtime typically requires additional dynamic libraries, differing from the main Interpreter path.

Type	Example	Purpose
Dispatch	<code>libLiteRtDispatch_Aml.so</code>	CompiledModel runtime dispatch
Compiler Plugin	<code>libLiteRtCompilerPlugin_Aml.so</code>	Compilation/graph building plugin
本地 case	<code>aml_CompiledModel_benchmark</code>	CompiledModel path validation